

Step 8 One-Pager — Safe Exploitation

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Problem. Step 7 built the *sensor* — an opponent model — and showed its danger: best-responding to an imperfect model can open a leak in your own play (the continuous model *lost* to Nash on Leduc). Step 8 builds the *actuator*: turn a model into profit **without becoming exploitable**. This is the second half of the thesis’s Behavioral Adaptation Framework (Contribution #1) and the launch point for multi-agent safe exploitation (Contribution #2).

Approach. Every safe-exploitation method is the same program — *maximize expected value against the opponent model, subject to a safety floor on your worst-case value* — which is linear in the hero’s sequence-form realization plan. One LP engine solves it by **constraint generation** (solve $\rightarrow$ read policy $\rightarrow$ call an exact best response as the worst-case oracle $\rightarrow$ add a cut if unsafe $\rightarrow$ re-solve), reusing Step 7’s engines, best-response, Nash and opponent zoo. Five families differ only in the floor: **RNR** (tunable p), **Ganzfried** (Nash value v^*), **prime-safe** ($v^* - \epsilon$ for an ϵ -equilibrium baseline), **SES** (blueprint value, enforced locally on a subgame via a gadget), **adaptation** (no more exploitable than the blueprint). Testbeds: Kuhn and Leduc, both exactly solvable, so every result is bracketed by exact analytical references. **All numbers below are measured.**

Key results (measured).

- *Safe exploitation works on Kuhn. Ganzfried is safe against every opponent* (worst-case v^* within $1e-3$) **and beats Nash’s own EV on every exploitable type** — e.g. **+0.222 vs Nash’s +0.146** against AlwaysPass. Full best response earns more (+0.975) but its worst-case collapses to **−0.5** (ruinously exploitable). This is the central validated result.
- *is measured, not fabricated.* Prime-safe/adaptation lower the floor to $v^* - \epsilon$ with the baseline’s exploitability measured at $\epsilon = \mathbf{0.0074}$, and earn a little more than Ganzfried by spending that budget (+0.266 vs +0.222 on AlwaysPass).
- *Canonical RNR is bang-bang, not a smooth frontier* (a contradicted prediction). In a game this small the max-min LP jumps from the safe vertex straight to full best response at $p = 0.7$ — the smooth curve is the *naive blend*’s, and it is dominated at the safe corner.
- *Headline — global safe-exploitation does not scale; local does.* On **Leduc**, the global solvers (Ganzfried/prime-safe/adaptation) **fail to converge within a 40-iteration cap** (worst-case **−0.64 to −1.33**, grossly unsafe), while the **subgame method (SES) converges** (194–350 iters) and stays near-safe (worst-case **−0.13**) while extracting **+0.25 to +0.68** vs weak types. This is the global-vs-local safety theory $\rightarrow$ practice gap, measured on a tiny game.
- *The safety guarantee bites against a worst-case adversary, not a benign one.* In the teaching attack (bait $\rightarrow$ Nash reveal), the honest separating signal is the safety-violation count (**full_br 40/40 refits, Ganzfried 0/40**), not realized profit — a gentle Nash “revealer” never claws back full_br’s bait-phase windfall. A punishing test needs an adaptive counter-exploiter.

Thesis connection. Step 7 = sensor; Step 8 = actuator. The Kuhn results confirm the actuator is sound; the Leduc non-convergence is the concrete argument for real-time subgame methods (SES / OX-Search) and/or an exact dual-LP formulation, and it is the empirical bridge into the scalable, multi-agent safe exploitation of Contribution #2.

Open questions. Scalable safety (exact dual-LP vs local/subgame — the Leduc wall); the SES gadget’s residual exploitability ($0.04 > \text{tol}$ — provably or only approximately safe?); a punishing teaching attack (adaptive reveal); and **N-player safety** — every guarantee here rests on the two-player zero-sum fact that Nash secures v^* against any opponent, an anchor that vanishes for $N > 2$ (Contribution #2).